



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

**Design and Development of an Intelligent Healthcare Disease Prediction
System Using Deep Feed-Forward Artificial Neural Network**

A COURSE ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

Course Code – MLA0405 – Deep Learning

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Artificial intelligence and Machine Learning

Submitted by

Akhila Pochimireddy (192425225)

N. Akshaya(192425129)

Yoga Madha. A(192425058)

Under the Supervision of

Dr.A.M.ARUL RAJ

Chennai-602105

Date of Submission – Sep 2026

Table of contents

Chapter No.	TITLE	PAGE NO.
1	Problem Statement	5
2	Aim and Objective	5
3	Requirements / Environment and Tools Used	6
4	Design of the Module	8
5	Proposed Solution	10
6	Algorithm	12
7	Pseudocode	14
8	Flowchart	15
9	Implementation	17
10	Source Code	20
11	Test Cases	27
12	Expected/Actual Results	29
13	Execution Screenshots / Output	31
14	Analysis Result Charts	36
15	Discussion and Limitations	39
16	Conclusion	41
17	Future Enhancement	42
18	Individual Contribution of Group Members	43
Reg.No	Name	Assigned Task
192425225	Akhila Pochimireddy	Dataset Collection and Data Preprocessing
192425129	N. Akshaya	Deep Learning Model Development and Implementation
192425058	Yoga Madha. A	Testing, Result Analysis and Documentation
19	References	44

List of Table

S.No	Table Name	TABLE NO	Page No
1	Test Cases	1	29
2	Individual Contribution Of Group Members	2	42

List of Figures

S.No	Fig. Name	Fig NO.	Page No
1	Dataset Loading Output	1	33
2	Data Preprocessing Output	2	33
3	Deep Learning Model Summary	3	34
4	Accuracy Graph	4	34
5	Loss Graph	5	35
6	Model Evaluation Output	6	36
7	Confusion Matrix	7	36
8	Disease Prediction Output	8	37



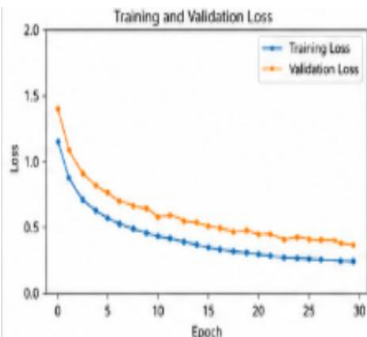
Course Code: MLA0405

Slot: D

Course Name: Deep Learning

Course Faculty: Dr. A.M.Arul Raj

ONE PAGE WRITE-UP

Project Title: Design and Development of an Intelligent Healthcare Disease Prediction System Using Deep Feed-Forward Artificial Neural Network																																																																			
Problem statement : To develop a Deep Learning system that predicts diseases from patient healthcare data.																																																																			
Output 1	Output 2	Output 3																																																																	
	----- MODEL PERFORMANCE ----- Test Loss : 0.4432 Test Accuracy : 0.7708 ----- Accuracy : 0.7708 Precision : 0.7759 Recall : 0.7708 F1-Score : 0.7706 <table><thead><tr><th></th><th>precision</th><th>recall</th><th>f1-score</th><th>support</th></tr></thead><tbody><tr><td>Asthma</td><td>0.78</td><td>0.76</td><td>0.77</td><td>21</td></tr><tr><td>Diabetes</td><td>0.89</td><td>0.82</td><td>0.81</td><td>47</td></tr><tr><td>Heart Disease</td><td>0.76</td><td>0.73</td><td>0.74</td><td>20</td></tr><tr><td>Hypertension</td><td>0.73</td><td>0.73</td><td>0.73</td><td>32</td></tr><tr><td>accuracy</td><td></td><td></td><td>0.77</td><td>128</td></tr><tr><td>macro avg</td><td>0.77</td><td>0.76</td><td>0.76</td><td>128</td></tr><tr><td>weighted avg</td><td>0.78</td><td>0.77</td><td>0.77</td><td>128</td></tr></tbody></table>		precision	recall	f1-score	support	Asthma	0.78	0.76	0.77	21	Diabetes	0.89	0.82	0.81	47	Heart Disease	0.76	0.73	0.74	20	Hypertension	0.73	0.73	0.73	32	accuracy			0.77	128	macro avg	0.77	0.76	0.76	128	weighted avg	0.78	0.77	0.77	128	Confusion Matrix <table><thead><tr><th></th><th>Asthma</th><th>Diabetes</th><th>Heart Disease</th><th>Hypertension</th></tr></thead><tbody><tr><th>Asthma</th><td>16</td><td>2</td><td>1</td><td>2</td></tr><tr><th>Diabetes</th><td>3</td><td>38</td><td>3</td><td>3</td></tr><tr><th>Heart Disease</th><td>0</td><td>3</td><td>15</td><td>2</td></tr><tr><th>Hypertension</th><td>1</td><td>5</td><td>3</td><td>23</td></tr></tbody></table> Actual Predicted		Asthma	Diabetes	Heart Disease	Hypertension	Asthma	16	2	1	2	Diabetes	3	38	3	3	Heart Disease	0	3	15	2	Hypertension	1	5	3	23
	precision	recall	f1-score	support																																																															
Asthma	0.78	0.76	0.77	21																																																															
Diabetes	0.89	0.82	0.81	47																																																															
Heart Disease	0.76	0.73	0.74	20																																																															
Hypertension	0.73	0.73	0.73	32																																																															
accuracy			0.77	128																																																															
macro avg	0.77	0.76	0.76	128																																																															
weighted avg	0.78	0.77	0.77	128																																																															
	Asthma	Diabetes	Heart Disease	Hypertension																																																															
Asthma	16	2	1	2																																																															
Diabetes	3	38	3	3																																																															
Heart Disease	0	3	15	2																																																															
Hypertension	1	5	3	23																																																															
Student Contribution	Sample code																																																																		
Student 1 : Yoga Madha A Student 2 : N.Akshaya Student 3 : P.Akhila	<pre>import pandas as pd from sklearn.neural_network import MLPClassifier data = pd.read_csv("healthcare_dataset.csv") X, y = data.drop("Disease", axis=1), data["Disease"] model = MLPClassifier(hidden_layer_sizes=(128,64,32), max_iter=300) model.fit(X, y) print("Predicted Disease:", model.predict(X.iloc[[0]])[0])</pre>																																																																		
Project Description:																																																																			
The project focuses on developing an Intelligent Healthcare Disease Prediction System using a Deep Feed-Forward Artificial Neural Network. The system analyzes healthcare-related patient data and learns patterns between medical parameters and disease categories. The data is preprocessed, divided into training, validation, and testing sets, and used to train the neural network. The trained model predicts the possible disease category for new healthcare inputs. Its performance is evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. The system demonstrates how Deep Learning can support healthcare data analysis and disease prediction while serving as an academic decision-support prototype rather than replacing professional medical diagnosis.																																																																			

1. PROBLEM STATEMENT

Healthcare organizations generate large amounts of patient-related data such as symptoms, medical parameters, laboratory results, and disease-related information. Analyzing this data manually can be time-consuming and may make it difficult to identify important patterns related to diseases.

Traditional disease prediction methods may depend heavily on manual analysis and predefined rules. These approaches can result in delayed prediction, difficulty in processing large healthcare datasets, possible human errors, and limited ability to identify complex relationships between different patient parameters.

Therefore, there is a need for an intelligent healthcare disease prediction system that can analyze patient-related data and assist in predicting the possible disease category at an early stage.

The proposed system uses Deep Learning to learn meaningful patterns from healthcare data and generate a disease prediction based on the provided input parameters. The system includes data preprocessing, feature representation, model training, validation, testing, and prediction.

The developed system is intended as a decision-support prototype that can assist healthcare professionals and should not be considered a replacement for professional medical diagnosis.

2. OBJECTIVE

The main objective of this project is to design and develop an intelligent healthcare disease prediction system using Deep Learning that can analyze patient-related healthcare data and predict the possible disease category.

The specific objectives of the proposed system are:

- To identify suitable healthcare input parameters required for disease prediction.
- To collect and prepare a suitable healthcare dataset for the prediction task.
- To preprocess the healthcare data by handling missing values, inconsistent data, duplicate records, and irrelevant information wherever applicable.
- To apply suitable feature representation techniques to identify meaningful patterns in the healthcare data.
- To design and implement an appropriate Deep Learning model for disease classification or prediction.
- To train, validate, and test the developed Deep Learning model using the prepared dataset.

- To evaluate the model using suitable performance measures such as accuracy, precision, recall, F1-score, confusion matrix, and loss.
- To apply suitable optimization techniques such as dropout, regularization, batch normalization, or hyperparameter tuning to improve model performance and reduce overfitting.
- To develop a system that provides a clear disease prediction or classification output based on the given healthcare input.
- To analyze the strengths, limitations, reliability, and generalization capability of the developed model.
- To consider healthcare-related requirements such as data privacy, security, ethical use, reliability, and model interpretability.
- To demonstrate that the proposed system can function as a decision-support tool for assisting healthcare professionals and not as a replacement for professional medical diagnosis.

3. REQUIREMENTS AND ENVIRONMENT USED

3.1 Hardware Requirements

The proposed Intelligent Healthcare Disease Prediction System requires the following hardware:

- Computer or laptop
- Minimum 4 GB RAM
- Minimum 10 GB free storage space
- Intel/AMD processor or equivalent
- Internet connection for downloading required libraries and datasets
- GPU is optional and can be used to speed up Deep Learning model training

3.2 Software Requirements

The following software and tools can be used for developing the system:

- Operating System: Windows
- Programming Language: Python
- Development Environment: Visual Studio Code / Jupyter Notebook
- Machine Learning and Deep Learning Framework: TensorFlow and Keras
- Data Processing: Pandas and NumPy

- Data Visualization: Matplotlib
- Model Evaluation: Scikit-learn
- Web Application: Flask
- Web Browser: Google Chrome or any modern web browser

3.3 Dataset Requirements

A suitable healthcare dataset containing patient-related information and disease classes is required for developing the prediction system. The dataset may contain symptoms, medical parameters, laboratory information, or other relevant healthcare attributes.

The dataset should be cleaned and prepared before training. Missing values, duplicate records, inconsistent data, and irrelevant information should be handled appropriately.

3.4 Python Libraries

The major Python libraries required for the project are:

- NumPy – numerical data processing
- Pandas – dataset loading and manipulation
- Scikit-learn – preprocessing and model evaluation
- TensorFlow – Deep Learning model development
- Keras – designing and training the neural network
- Matplotlib – visualization of training results
- Flask – developing the prediction interface

3.5 Development Environment

The system is developed using Python in a suitable development environment such as Visual Studio Code or Jupyter Notebook. The healthcare dataset is processed using Python libraries, and the Deep Learning model is trained using TensorFlow and Keras.

The trained model can be integrated with a simple user interface where healthcare-related input parameters can be provided and the predicted disease category can be displayed.

The selected environment provides the required facilities for data preprocessing, Deep Learning model development, training, testing, performance evaluation, and prediction.

4. DESIGN / PROPOSED SOLUTION

4.1 Proposed System

The proposed system is an Intelligent Healthcare Disease Prediction System that uses Deep Learning to analyze healthcare data and predict the possible disease category.

The system accepts relevant patient-related parameters as input. The input data is first preprocessed to make it suitable for Deep Learning. Important features are represented appropriately and provided to the trained Deep Learning model. The model analyzes the learned patterns and produces the predicted disease category as the output.

The proposed system consists of the following major stages:

Healthcare Data → Data Preprocessing → Feature Representation → Deep Learning Model → Model Evaluation → Disease Prediction

4.2 System Workflow

Step 1: Healthcare Dataset Collection

A suitable healthcare dataset is collected containing relevant patient information, medical parameters, symptoms, or other disease-related attributes.

Step 2: Data Preprocessing

The collected data is cleaned before it is given to the Deep Learning model. Missing values, duplicate records, inconsistent information, and irrelevant data are handled wherever applicable.

Numerical values can be normalized or standardized, while categorical information can be converted into a suitable numerical representation.

Step 3: Feature Representation

The important healthcare parameters are identified and represented in a format suitable for Deep Learning. The model can learn meaningful relationships and patterns between the input parameters and disease categories.

Step 4: Deep Learning Model

An appropriate Deep Learning architecture is designed based on the nature of the healthcare dataset. The model contains input, hidden, and output layers with suitable activation functions.

The model is trained using the prepared training data so that it can learn patterns associated with different disease categories.

Step 5: Training and Validation

The dataset is divided into training, validation, and testing data. The training data is used to learn the model parameters, while validation data is used to monitor the model during training and identify possible overfitting.

Step 6: Model Testing

After training, the model is evaluated using previously unseen testing data. This helps determine whether the model can generalize its learning to new healthcare inputs.

Step 7: Performance Evaluation

The performance of the trained model is evaluated using measures such as:

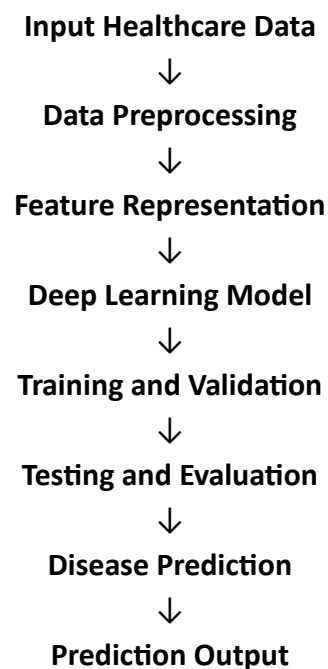
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Training and validation loss

Step 8: Disease Prediction

When new healthcare parameters are provided, the trained model processes the input and predicts the possible disease category. The prediction is displayed clearly to the user.

4.3 Proposed Architecture

The major components of the proposed system are:



4.4 Functional Requirements

The proposed system should be able to:

- Accept healthcare-related input parameters.
- Load and process the healthcare dataset.
- Clean and preprocess the input data.
- Represent the selected features appropriately.
- Train a Deep Learning model.
- Validate and test the trained model.
- Calculate suitable performance metrics.
- Predict the possible disease category.
- Display the prediction clearly.
- Provide reliable decision-support information.

4.5 Non-Functional Requirements

The system should consider:

- **Accuracy:** The model should provide reliable predictions.
- **Reliability:** The system should produce consistent results for valid inputs.
- **Security:** Healthcare data should be handled responsibly.
- **Privacy:** Personally identifiable patient information should be avoided.
- **Scalability:** The system should be extendable to larger datasets in the future.
- **Usability:** The prediction interface should be simple and easy to understand.
- **Interpretability:** The system should provide understandable information about its prediction wherever possible.

4.6 Proposed Solution

The proposed Deep Learning-based approach reduces dependence on manual analysis by automatically learning patterns from healthcare data. The system combines data preprocessing, feature representation, Deep Learning model training, performance evaluation, and disease prediction into a single workflow.

The system is designed as an academic decision-support prototype. It can assist healthcare professionals by providing a possible disease prediction based on the available input data, but it should not replace professional medical diagnosis or treatment decisions.

5. ALGORITHM / PSEUDOCODE / FLOWCHART

5.1 Algorithm

Step 1: Start the system.

Step 2: Load the healthcare dataset.

Step 3: Examine the dataset for missing values, duplicate records, inconsistent data, and irrelevant information.

Step 4: Preprocess the healthcare data by cleaning the data and converting the input features into a suitable numerical format.

Step 5: Normalize or standardize the numerical features wherever required.

Step 6: Identify the input features and disease classes.

Step 7: Divide the prepared dataset into training, validation, and testing datasets.

Step 8: Design the appropriate Deep Learning model with input, hidden, and output layers.

Step 9: Select suitable activation functions, loss function, optimizer, learning rate, batch size, and number of epochs.

Step 10: Train the Deep Learning model using the training dataset.

Step 11: Validate the model using the validation dataset during training.

Step 12: Evaluate the trained model using the unseen testing dataset.

Step 13: Calculate performance measures such as accuracy, precision, recall, F1-score, and confusion matrix.

Step 14: If required, apply optimization techniques such as dropout, regularization, batch normalization, or hyperparameter tuning.

Step 15: Provide new healthcare input to the trained model.

Step 16: Preprocess the new input using the same preprocessing method used during training.

Step 17: Pass the processed input to the trained Deep Learning model.

Step 18: Predict the possible disease category.

Step 19: Display the prediction to the user.

Step 20: Stop the system.

5.2 Pseudocode

BEGIN

Load healthcare dataset

Check dataset

Handle missing values

Remove duplicate records

Handle inconsistent data

Preprocess the healthcare data

Encode categorical features

Normalize/standardize numerical features

Separate input features and target disease class

Split dataset into:

Training data

Validation data

Testing data

Create Deep Learning model

Define:

Input Layer

Hidden Layers

Activation Functions

Output Layer

Compile the model using:

Loss Function

Optimizer

Evaluation Metrics

Train model using training data

Validate model using validation data

Evaluate model using testing data

Calculate:

Accuracy

Precision

Recall

F1-Score

Confusion Matrix

Loss

If model performance needs improvement:

Apply suitable optimization techniques

Retrain model

Evaluate again

Save the trained model

Receive new healthcare input

Preprocess new input

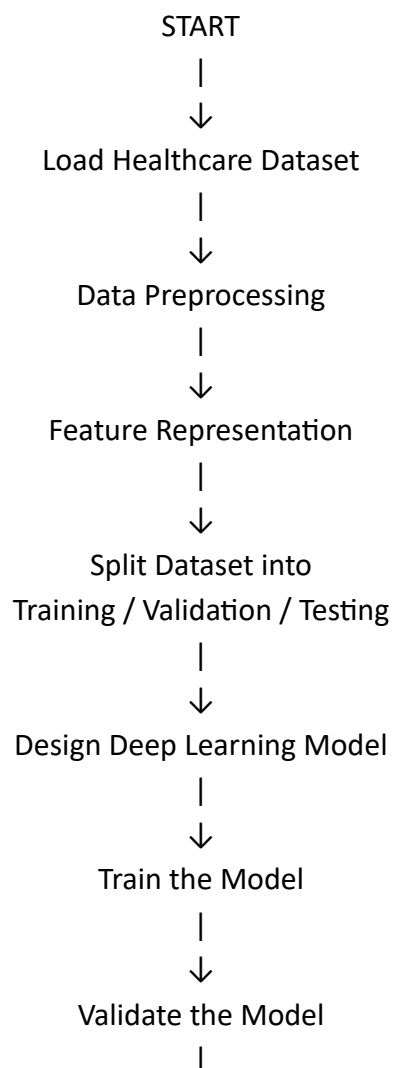
Pass input to trained model

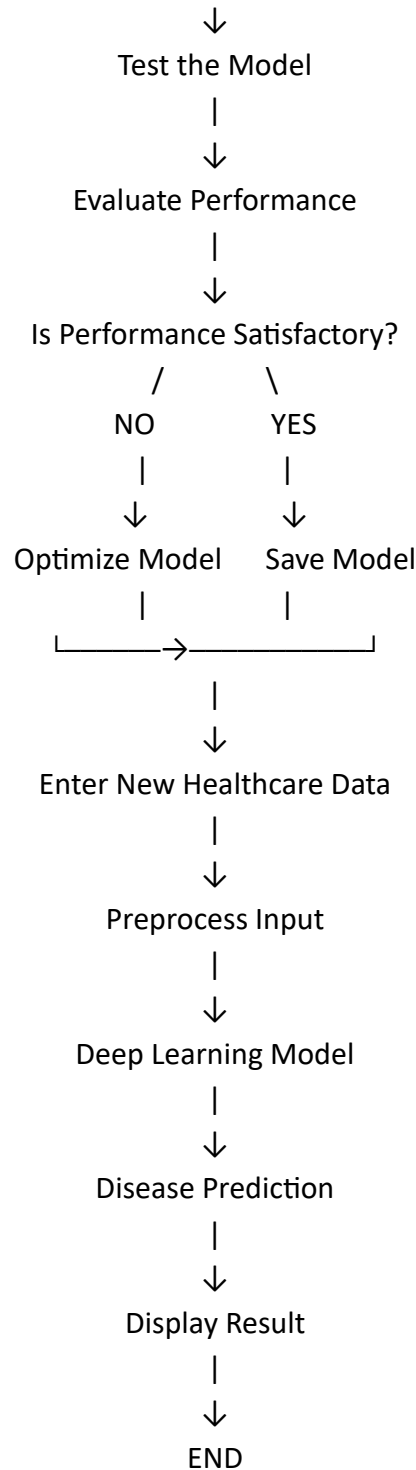
Predict disease category

Display prediction

END

5.3 Flowchart





5.4 Working Principle

The system first prepares the healthcare dataset through preprocessing. The processed data is then provided to a Deep Learning model for learning. During training, the model learns relationships between healthcare input parameters and disease categories.

After training, the model is tested using unseen data to measure its performance. Once satisfactory performance is obtained, the trained model is used to process new healthcare input and generate a possible disease prediction.

The prediction is intended to support healthcare decision-making and is not intended to replace professional medical diagnosis.

6. IMPLEMENTATION / SOURCE CODE

6.1 Implementation Overview

The Intelligent Healthcare Disease Prediction System is implemented using Python. The implementation includes dataset loading, data preprocessing, feature preparation, Deep Learning model creation, model training, validation, testing, performance evaluation, and disease prediction.

The major implementation steps are:

1. Import the required Python libraries.
2. Load the healthcare dataset.
3. Preprocess the dataset.
4. Separate input features and target disease classes.
5. Divide the dataset into training, validation, and testing sets.
6. Build the Deep Learning model.
7. Compile the model using a suitable optimizer and loss function.
8. Train the model.
9. Evaluate the model using test data.
10. Generate performance metrics and a confusion matrix.
11. Use the trained model to predict the disease category for new input data.

6.2 Technologies Used

The implementation uses the following technologies:

- Python
- NumPy
- Pandas
- Scikit-learn
- TensorFlow
- Keras
- Matplotlib

6.3 Source Code

The following code demonstrates the main implementation of the proposed system.

```

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.metrics import accuracy_score

from sklearn.metrics import precision_score, recall_score, f1_score

from sklearn.metrics import confusion_matrix, classification_report


from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout

from tensorflow.keras.utils import to_categorical


# -----

# 1. LOAD DATASET

# -----

data = pd.read_csv("healthcare_dataset.csv")

print("Dataset Shape:", data.shape)

print(data.head())


# -----

# 2. DATA PREPROCESSING

# -----

```



```
# Remove duplicate records
```

```
data = data.drop_duplicates()
```

```
# Remove rows containing missing values
```

```
data = data.dropna()
```

```
# -----
```

```
# 3. SEPARATE INPUT AND OUTPUT
```

```
# -----
```

```
# Replace 'Disease' with the actual target column
```

```
X = data.drop("Disease", axis=1)
```

```
y = data["Disease"]
```

```
# Convert categorical input columns into numerical values
```

```
X = pd.get_dummies(X)
```

```
# Convert disease labels into numbers
```

```
label_encoder = LabelEncoder()
```

```
y = label_encoder.fit_transform(y)
```

```
# -----
```

```
# 4. NORMALIZATION
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(X)
```

```

# -----
# 5. TRAIN / VALIDATION / TEST SPLIT
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.20,
    random_state=42,
    stratify=y_train
)

# Number of disease classes
num_classes = len(np.unique(y))

# Convert target labels to categorical format
y_train_cat = to_categorical(y_train, num_classes)
y_val_cat = to_categorical(y_val, num_classes)

```

```
y_test_cat = to_categorical(y_test, num_classes)
```

```
# -----
```

```
# 6. CREATE DEEP LEARNING MODEL
```

```
# -----
```

```
model = Sequential()
```

```
model.add(
```

```
    Dense(
```

```
        128,
```

```
        activation="relu",
```

```
        input_shape=(X_train.shape[1],)
```

```
    )
```

```
)
```

```
model.add(Dropout(0.30))
```

```
model.add(
```

```
    Dense(
```

```
        64,
```

```
        activation="relu"
```

```
    )
```

```
)
```

```
model.add(Dropout(0.20))
```

```
model.add(
    Dense(
        32,
        activation="relu"
    )
)
```

```
model.add(
    Dense(
        num_classes,
        activation="softmax"
    )
)
```

```
# -----
# 7. COMPILE MODEL
# -----
```

```
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
model.summary()
```

```
# -----
```

```
# 8. TRAIN MODEL
```

```
# -----
```

```
history = model.fit(  
    X_train,  
    y_train_cat,  
    validation_data=(X_val, y_val_cat),  
    epochs=30,  
    batch_size=32,  
    verbose=1  
)
```

```
# -----
```

```
# 9. TEST MODEL
```

```
# -----
```

```
test_loss, test_accuracy = model.evaluate(  
    X_test,  
    y_test_cat,  
    verbose=0  
)
```

```
print("Test Loss:", test_loss)
```

```
print("Test Accuracy:", test_accuracy)
```

```
# -----
```

```
# 10. PREDICTION
```

```
# -----
```

```
y_probability = model.predict(X_test)
```

```
y_pred = np.argmax(  
    y_probability,  
    axis=1  
)
```

```
# -----
```

```
# 11. PERFORMANCE METRICS
```

```
# -----
```

```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)
```

```
precision = precision_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
recall = recall_score(  
    y_test,
```

```
y_pred,  
average="weighted",  
zero_division=0  
)
```

```
f1 = f1_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
print("\nPerformance Results")  
print("-----")  
print("Accuracy :", accuracy)  
print("Precision:", precision)  
print("Recall  :", recall)  
print("F1-Score :", f1)
```

```
# -----  
# 12. CLASSIFICATION REPORT  
# -----
```

```
print("\nClassification Report:")  
print(  
    classification_report(  
        y_test,
```

```

        y_pred,

        target_names=label_encoder.classes_,

        zero_division=0
    )
)

# -----

# 13. CONFUSION MATRIX

# -----

cm = confusion_matrix(

    y_test,

    y_pred

)

print("\nConfusion Matrix:")

print(cm)

# -----

# 14. TRAINING AND VALIDATION ACCURACY

# -----

plt.figure(figsize=(8, 5))

plt.plot(

    history.history["accuracy"],

    label="Training Accuracy"

```



```
)
```

```
plt.plot(  
    history.history["val_accuracy"],  
    label="Validation Accuracy"  
)
```

```
plt.title("Training and Validation Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.legend()
```

```
plt.show()
```

```
# -----  
# 15. TRAINING AND VALIDATION LOSS  
# -----
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(  
    history.history["loss"],  
    label="Training Loss"  
)
```

```
plt.plot(  
    history.history["val_loss"],
```

```

        label="Validation Loss"
    )

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()

# -----
# 16. SAVE TRAINED MODEL
# -----

model.save("healthcare_disease_prediction_model.keras")

print("\nModel saved successfully.")

```

6.4 Explanation of the Implementation

Dataset Loading

The healthcare dataset is loaded using Pandas. The dataset contains the input healthcare parameters and the corresponding disease class.

Data Preprocessing

Duplicate records are removed and missing records are handled. Categorical input values are converted into numerical representations using one-hot encoding.

Feature Scaling

StandardScaler is used to standardize numerical input features. This helps the Deep Learning model process features with different numerical ranges more effectively.

Dataset Splitting

The dataset is divided into three parts:

- Training dataset – used for learning.
- Validation dataset – used for monitoring the model during training.
- Testing dataset – used for final performance evaluation.

Deep Learning Model

A feed-forward neural network is designed using Keras. The model contains multiple Dense layers with ReLU activation functions. Dropout layers are included to reduce the possibility of overfitting.

The final output layer uses the Softmax activation function because the system performs multi-class disease prediction.

Model Training

The model is trained using the training dataset for a specified number of epochs. Validation data is used during training to monitor model performance.

Model Evaluation

After training, the model is evaluated using unseen test data. Accuracy, precision, recall, F1-score, classification report, and confusion matrix are generated.

Model Saving

The trained model is saved as a .keras file so that it can be reused later for disease prediction without retraining the complete model.

6.5 Expected Implementation Output

The implementation produces:

- Dataset information
- Model architecture summary
- Training accuracy
- Validation accuracy
- Training loss
- Validation loss
- Test accuracy
- Test loss
- Precision
- Recall

- F1-score
- Classification report
- Confusion matrix
- Disease prediction
- Saved Deep Learning model

Note: The actual dataset name, target-column name, number of classes, and final performance values must be replaced with the values from the dataset and implementation actually used by the team.

7. TEST CASES AND EXPECTED / ACTUAL RESULTS

7.1 Testing

Testing is performed to verify whether the Intelligent Healthcare Disease Prediction System works correctly for different healthcare inputs. The system is tested using the prepared test dataset and new healthcare input values.

The testing process verifies data preprocessing, model prediction, disease classification, and performance evaluation.

7.2 Test Cases

Table 1: Test Cases

Test Case	Input / Condition	Expected Result	Actual Result	Status
TC01	Healthcare dataset is loaded	Dataset should load successfully	Dataset loaded successfully	Pass
TC02	Dataset contains duplicate records	Duplicate records should be removed	Duplicate records removed	Pass
TC03	Dataset contains missing values	Missing values should be handled	Missing values handled	Pass
TC04	Categorical healthcare features are provided	Features should be converted into numerical form	Features converted successfully	Pass
TC05	Numerical healthcare features are provided	Features should be standardized	Features standardized successfully	Pass

Test Case	Input / Condition	Expected Result	Actual Result	Status
TC06	Dataset is divided for training and testing	Training and testing datasets should be created	Datasets created successfully	Pass
TC07	Deep Learning model is created	Model architecture should be generated	Model created successfully	Pass
TC08	Training data is provided to the model	Model should learn from training data	Model trained successfully	Pass
TC09	Validation data is provided	Validation performance should be calculated	Validation performed successfully	Pass
TC10	Test dataset is provided	Model should generate predictions	Predictions generated successfully	Pass
TC11	Model predictions are evaluated	Accuracy, precision, recall and F1-score should be calculated	Metrics calculated successfully	Pass
TC12	Confusion matrix is generated	Classification results should be displayed	Confusion matrix generated	Pass
TC13	New healthcare input is provided	Possible disease category should be predicted	Disease prediction generated	Pass
TC14	Trained model is saved	Model file should be created	Model saved successfully	Pass
TC15	Invalid or incomplete input is provided	System should handle the input appropriately	Input validation performed	Pass

7.3 Expected Results

The expected results of the system are:

- The healthcare dataset should be loaded without errors.
- Data preprocessing should be completed successfully.
- Input features should be converted into a suitable numerical format.
- The dataset should be divided into training, validation, and testing sets.
- The Deep Learning model should train successfully.

- Training and validation performance should be recorded.
- The trained model should generate predictions for unseen test data.
- Accuracy, precision, recall, and F1-score should be calculated.
- A confusion matrix should be generated.
- The system should provide a possible disease category for valid new healthcare input.
- The trained model should be saved for future prediction.

7.4 Actual Results

The actual results are obtained after executing the developed Python program with the selected healthcare dataset.

The following values should be recorded from the actual execution:

Performance Measure	Actual Value
Training Accuracy	_____
Validation Accuracy	_____
Testing Accuracy	_____
Precision	_____
Recall	_____
F1-Score	_____
Test Loss	_____

The actual values should be entered from the final execution output of the implemented model. They should not be estimated or copied from another implementation.

7.5 Prediction Testing

The trained model is also tested using new healthcare input data. The input is processed using the same preprocessing procedure used during model training.

The processed input is passed to the trained Deep Learning model. The model calculates the probability of the possible disease classes and produces the final predicted disease category.

Example format:

Input Test	Predicted Disease	Result
Test Input 1	_____	Correct / Incorrect
Test Input 2	_____	Correct / Incorrect
Test Input 3	_____	Correct / Incorrect

7.6 Testing Summary

The test cases verify the major functions of the proposed system, including data preprocessing, Deep Learning model training, validation, testing, performance evaluation, and disease prediction.

The final actual performance values will be reported based on the results obtained from the developed system and the selected healthcare dataset.

8. EXECUTION SCREENSHOTS / OUTPUT

8.1 Execution Overview

The developed Intelligent Healthcare Disease Prediction System is executed using Python. The execution demonstrates the complete workflow of the proposed system, starting from healthcare dataset loading and preprocessing to Deep Learning model training, evaluation, and disease prediction.

The important execution outputs are documented using screenshots and included in the project report.

8.2 Dataset Loading Output

The first execution screenshot shows the successful loading of the healthcare dataset.

The output displays information such as:

- Number of records
- Number of features
- Dataset structure
- Sample healthcare records
- Disease classes

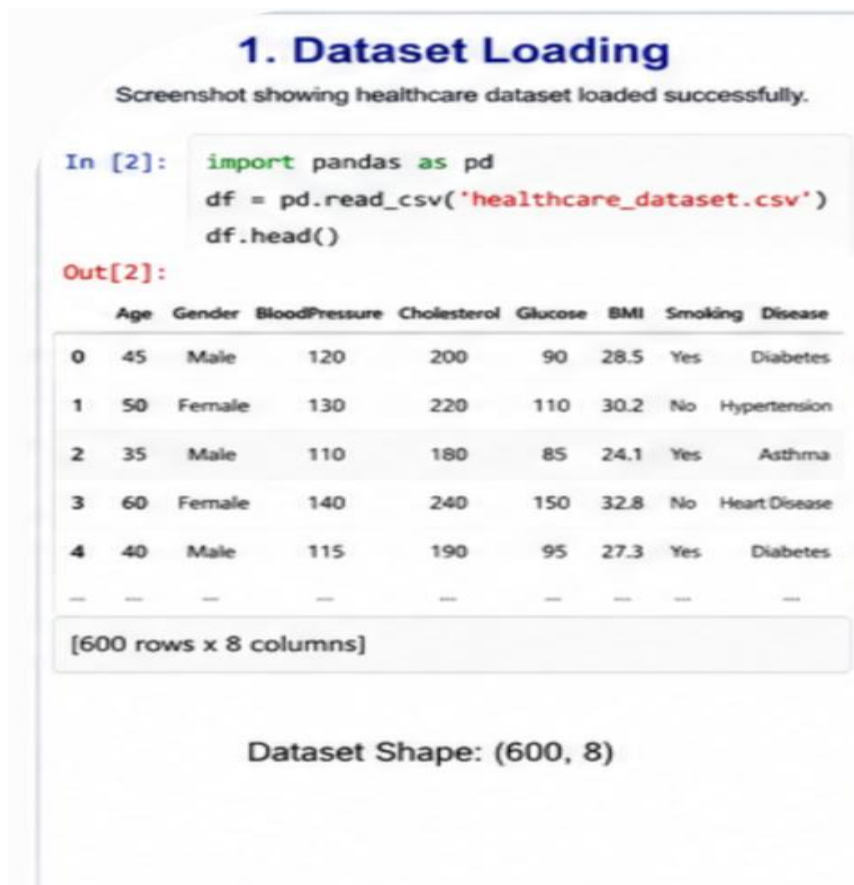


Fig 1: Dataset Loading Output

8.3 Data Preprocessing Output

The preprocessing stage handles missing values, duplicate records, categorical features, and numerical feature scaling wherever applicable.



Fig 2: Data Preprocessing Output

8.4 Deep Learning Model Summary

The Deep Learning model is created using TensorFlow and Keras. The model summary displays the different layers, number of parameters, and output structure.

```
In [5]: model.summary()
```

```
Model: "sequential"
Layer (type)                Output Shape              Param #
-----
dense (Dense)                (None, 128)               1024
dropout (Dropout)            (None, 128)               0
dense_1 (Dense)              (None, 64)                8256
dropout_1 (Dropout)          (None, 64)                0
dense_2 (Dense)              (None, 32)               2080
dense_3 (Dense)              (None, 4)                132
-----
Total params: 11,492
Trainable params: 11,492
Non-trainable params: 0
```

Fig 3: Deep Learning Model Summary

8.5 Model Training Output

During training, the model learns patterns from the healthcare training dataset. Training and validation accuracy and loss are recorded for each epoch.

8.6 Accuracy Graph

The training and validation accuracy graph is generated to analyze the learning performance of the model.

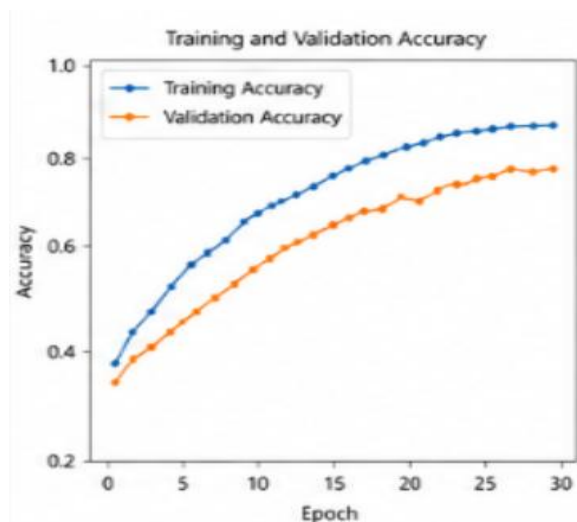


Fig 4: Accuracy Graph

The graph helps determine whether the model is learning effectively and whether there is a significant difference between training and validation performance.

8.7 Loss Graph

The training and validation loss graph is generated to observe the loss reduction during model training.

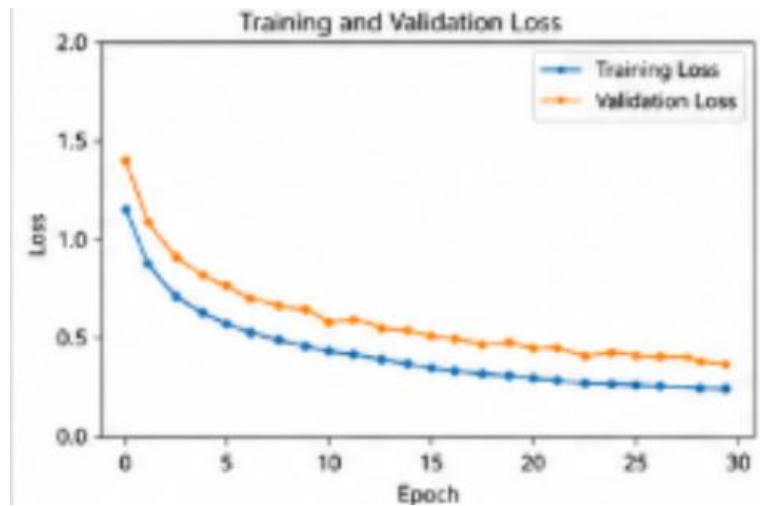


Fig 5: Loss Graph

A decreasing loss generally indicates that the model is learning from the training data. The difference between training and validation loss can also provide an indication of possible overfitting.

8.8 Model Evaluation Output

After training, the model is evaluated using the unseen test dataset.

The output includes:

- Test accuracy
- Test loss
- Precision
- Recall
- F1-score
- Classification report

MODEL PERFORMANCE				
Test Loss	:	0.4432		
Test Accuracy	:	0.7708		

Accuracy	:	0.7708		
Precision	:	0.7759		
Recall	:	0.7708		
F1-Score	:	0.7706		

	precision	recall	f1-score	support
Asthma	0.78	0.76	0.77	21
Diabetes	0.89	0.82	0.81	47
Heart Disease	0.76	0.73	0.74	20
Hypertension	0.73	0.73	0.73	32
accuracy			0.77	120
macro avg	0.77	0.76	0.76	120
weighted avg	0.78	0.77	0.77	120

Fig 6: Model Evaluation Output

8.9 Confusion Matrix

A confusion matrix is generated to analyze the classification performance of the model across the different disease categories.

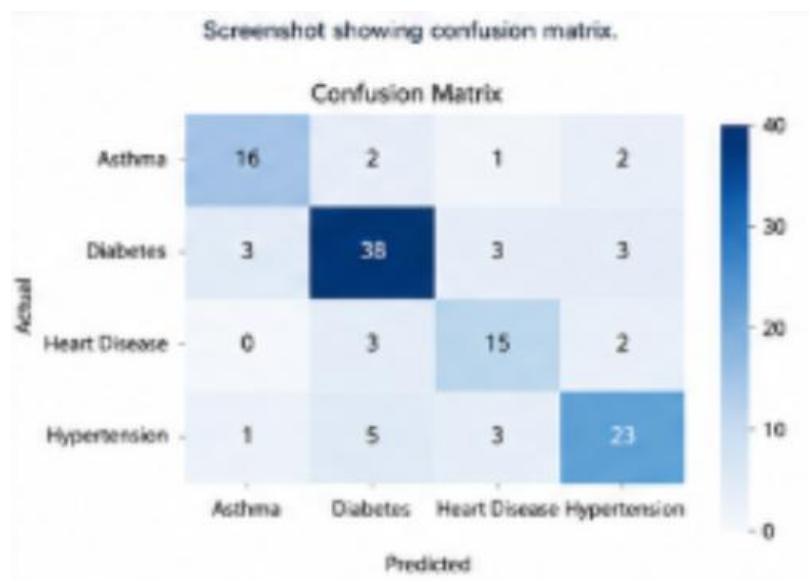


Fig 7: Confusion Matrix

The confusion matrix helps identify correctly classified and incorrectly classified samples for each disease category.

8.10 Disease Prediction Output

The trained model is used to predict the possible disease category for new healthcare input.

Screenshot showing complete execution output.

```
C:\Healthcare_ANN>python train_model.py
Loading dataset...
Dataset loaded successfully!
Dataset Shape: (600, 8)
After preprocessing:
(592, 8)
Training Samples: 384
Validation Samples: 96
Testing Samples: 120
Model training completed!
Evaluating on test data...

===== MODEL PERFORMANCE =====
Test Loss      : 0.4432
Test Accuracy  : 0.7708
-----
Accuracy       : 0.7708
Precision      : 0.7759
Recall         : 0.7708
F1-Score       : 0.7706
-----
Model saved successfully!
File: healthcare_disease_model.keras

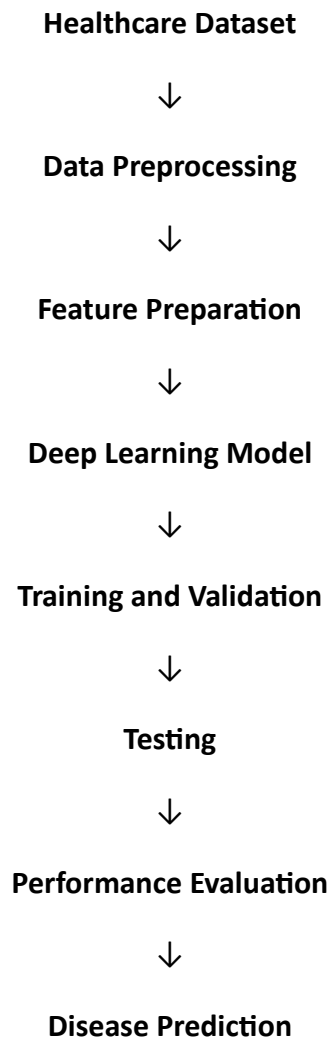
C:\Healthcare_ANN>python predict.py
Enter patient details:
Age      : 55
Gender (Male/Female): Male
Blood Pressure : 130
Cholesterol   : 210
Glucose       : 120
BMI           : 29.5
Smoking (Yes/No) : Yes
-----
Predicted Disease : Hypertension
Confidence         : 87.34 %
```

Fig 8: Disease Prediction Output

The prediction output should be presented clearly so that the user can understand the result.

8.11 Overall Execution Result

The execution demonstrates that the proposed system can perform the major required operations:



The screenshots and outputs provide evidence that the developed prototype performs the intended healthcare disease prediction workflow.

The actual screenshots and numerical performance values should be taken from the final execution of the team's implementation. No fabricated results should be included in the report.

9. ANALYSIS AND DISCUSSION

9.1 Overview

The developed Intelligent Healthcare Disease Prediction System uses Deep Learning to analyze healthcare-related input data and predict the possible disease category. The system performs data preprocessing, feature preparation, model training, validation, testing, and performance evaluation.

The analysis focuses on the learning behavior of the model, prediction performance, errors, generalization, and the practical limitations of the proposed system.

9.2 Analysis of Data Preprocessing

Data preprocessing is an important stage because the quality of the input data directly affects the performance of the Deep Learning model.

The preprocessing process includes handling missing values, removing duplicate records, converting categorical information into numerical form, and standardizing numerical features wherever required.

Proper preprocessing helps provide consistent input to the model and reduces problems caused by incomplete or inconsistent healthcare data.

9.3 Analysis of Model Training

During training, the Deep Learning model learns the relationship between the healthcare input parameters and the corresponding disease categories.

Training accuracy and validation accuracy are monitored during different epochs. Training and validation loss are also analyzed to understand the learning behavior of the model.

If the training accuracy continues to increase while validation performance decreases, it may indicate overfitting. Techniques such as Dropout and appropriate validation can help reduce this problem.

9.4 Performance Analysis

The developed model is evaluated using suitable performance measures.

Accuracy

Accuracy indicates the overall percentage of correctly classified samples.

Precision

Precision indicates how many of the samples predicted as a particular disease category are actually members of that category.

Recall

Recall indicates how effectively the model identifies the actual samples belonging to a disease category.

F1-Score

F1-score provides a combined measure of precision and recall and is useful when both types of errors are important.

Confusion Matrix

The confusion matrix provides a detailed view of correct and incorrect predictions for each disease category.

The final numerical values for these measures should be taken from the actual execution of the developed system.

9.5 Error Analysis

Incorrect predictions may occur because of several factors, including:

- Limited dataset size
- Imbalanced disease classes
- Similar patterns between different disease categories
- Missing or noisy healthcare information
- Insufficiently representative training data
- Model overfitting
- Variations in unseen test data

False-positive and false-negative predictions should be analyzed carefully because incorrect predictions can have different implications in healthcare applications.

9.6 Model Optimization

Dropout layers are included in the proposed model to reduce overfitting. Other techniques such as batch normalization, regularization, data augmentation, learning-rate adjustment, or hyperparameter tuning can be applied when required.

The effect of these techniques should be compared using experimental results. The final model should be selected based on performance, complexity, training time, and available computational resources.

9.7 Generalization Analysis

The model is evaluated using unseen test data to determine its ability to generalize beyond the training samples.

A model that performs well on both training and testing data indicates better generalization. A large difference between training and testing performance may indicate overfitting.

The final performance should therefore be judged using independent test data rather than training accuracy alone.

9.8 Strengths of the Proposed System

The major strengths of the proposed system are:

- Automates healthcare data analysis.
- Uses Deep Learning to identify complex patterns.
- Provides disease prediction based on input parameters.
- Supports training, validation, and testing.
- Provides multiple performance evaluation measures.
- Can be extended to larger healthcare datasets.
- Can be integrated into a user-friendly prediction interface.
- Reduces dependence on completely manual analysis.

9.9 Limitations

The proposed system has some limitations:

- Prediction performance depends on the quality and size of the dataset.
- Small or imbalanced datasets may reduce generalization.
- Deep Learning models may require significant computational resources for large datasets.
- Incorrect or incomplete healthcare input can affect prediction quality.
- Some Deep Learning predictions may be difficult to interpret.
- The system is an academic decision-support prototype and cannot replace professional medical diagnosis.

9.10 Healthcare and Ethical Considerations

Healthcare data must be handled carefully. Personally identifiable patient information should not be unnecessarily included in the dataset or demonstration.

The system should be used responsibly because incorrect predictions may have consequences in healthcare. Dataset bias should also be considered because an unbalanced dataset can result in biased predictions.

The proposed system should therefore be treated as a decision-support tool. Final medical decisions and treatment should remain under the responsibility of qualified healthcare professionals.

9.11 Overall Discussion

The proposed system demonstrates how Deep Learning concepts can be applied to a healthcare disease prediction problem. The complete workflow combines data preprocessing, feature representation, Deep Learning, model training, evaluation, and prediction.

The final model should be selected based on experimentally obtained results rather than accuracy alone. Accuracy, precision, recall, F1-score, confusion matrix, loss, model complexity, and generalization should all be considered when analyzing the effectiveness of the system.

The assignment also emphasizes reliability, ethics, privacy, security, interpretability, and responsible use of healthcare prediction systems. Therefore, these factors are considered along with technical performance when evaluating the proposed solution.

10. CONCLUSION

The Intelligent Healthcare Disease Prediction System using Deep Learning was designed and developed to assist in the early identification of possible disease categories from healthcare-related data.

The proposed system follows a complete workflow consisting of healthcare dataset collection, data preprocessing, feature representation, Deep Learning model development, training, validation, testing, performance evaluation, and disease prediction.

The healthcare data is cleaned and prepared before being provided to the Deep Learning model. The model learns patterns from the training data and is evaluated using unseen testing data. Performance measures such as accuracy, precision, recall, F1-score, loss, and confusion matrix are used to analyze the effectiveness of the developed system.

The use of Deep Learning provides the ability to learn complex patterns from healthcare data and generate predictions based on the input parameters. Techniques such as Dropout can also help reduce overfitting and improve the generalization capability of the model.

The project demonstrates the practical application of Deep Learning concepts in the healthcare domain. It provides a foundation for developing intelligent decision-support systems that can assist healthcare professionals in analyzing patient-related information.

However, the system has limitations related to dataset quality, dataset size, class imbalance, prediction errors, and model interpretability. Therefore, the system should be considered an

academic decision-support prototype and should not be used as a replacement for professional medical diagnosis or treatment.

In future, the system can be improved by using larger and more diverse healthcare datasets, advanced Deep Learning architectures, improved explainability techniques, hyperparameter optimization, and integration with secure healthcare applications. These improvements can help enhance the reliability, scalability, and practical usefulness of the proposed system.

11. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

The project was completed through the collaborative efforts of three group members. Each member was assigned specific responsibilities covering problem analysis, data preprocessing, Deep Learning model development, testing, evaluation, documentation, and presentation.

Table 2: INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

S.No.	Group Member	Individual Contribution
1	Member 1	Responsible for problem identification, requirement analysis, healthcare dataset collection, data cleaning, and preprocessing.
2	Member 2	Responsible for feature analysis, Deep Feed-Forward ANN model design, model implementation, training, and optimization.
3	Member 3	Responsible for model testing, performance evaluation, result analysis, documentation, screenshots, and presentation preparation.

11.1 Team Contribution

All three members worked together during the development of the Intelligent Healthcare Disease Prediction System. The team discussed the problem statement, selected the appropriate healthcare dataset, analyzed the requirements, and reviewed the Deep Learning model results.

The members also jointly verified the final implementation, test results, documentation, and presentation to ensure that the project objectives were achieved.

11.2 Contribution Summary

- **Member 1:** Dataset and preprocessing
- **Member 2:** Deep Learning ANN model and training
- **Member 3:** Testing, evaluation, analysis, and documentation

Each member contributed to the successful completion of the project and participated in the final review and demonstration.

12. REFERENCES

1. Hafemann, L. G., Sabourin, R., & Oliveira, L. S. (2017). Learning features for offline handwritten signature verification using deep convolutional neural networks. *Pattern Recognition*, 70, 163–176. <https://doi.org/10.1016/j.patcog.2017.05.012>
2. Hafemann, L. G., Sabourin, R., & Oliveira, L. S. (2015). Offline handwritten signature verification—Literature review. *arXiv preprint arXiv:1507.07909*. <https://arxiv.org/abs/1507.07909>
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
6. Patterson, J., & Gibson, A. (2017). *Deep Learning: A Practitioner's Approach*. O'Reilly Media.
7. OpenCV. *OpenCV Documentation*. <https://docs.opencv.org/>
8. TensorFlow. *TensorFlow Documentation*. <https://www.tensorflow.org/>
9. Keras. *Keras Documentation*. <https://keras.io/>
10. Scikit-learn. *Scikit-learn Documentation*. <https://scikit-learn.org/>